

AI 기반 주조 결함 검사 시스템

Casting Defect Detection System

EfficientNet-B0

Claude 4.5

Streamlit

2026년 1월

정혜원



Table of Contents

01	프로젝트 추진 배경	09	Orchestration 패턴 정의
02	주요 기술 스택	10	핵심 코드
03	데이터셋 및 분석 대상 소개	11	Streamlit 시스템 인터페이스
04	모델 비교 및 최종 모델 선정	12	프로젝트 회고
05	모델 구조 및 Transfer Learning		
06	Accuracy & Loss 분석		
07	Confusion Matrix 분석		
08	시스템 폴더 구조		

프로젝트 추진 배경

⚠ Traditional Method

육안 검사의 한계

🧑 검사자 피로도 및 집중력 저하
장시간 근무 시 정확도 급격히 하락

⚖ 주관적 판단의 불일치
검사자별 상이한 합격/불합격 기준

🔍 미세 결함 누락 위험
평균 **10-15%**의 결함 간과 가능성

🕒 처리 속도 및 비용 한계
수동 공정으로 인한 병목 현상 및 인건비 상승

VS

🤖 AI-Driven System

AI 검사의 혁신

∞ 일관된 품질 유지
피로도 없는 상시 최상위 성능 가동

✓ 객관적 데이터 기반 판단
정량적 수치에 의한 표준화된 검사

🎯 초정밀 결함 감지
미세 결함 **99% 이상** 탐지 정확도

⚡ 실시간 처리 및 자동화
1초 미만 처리 및 전 공정 이력 관리

주요 기술 스택

Vision Engine

Deep Learning

- | | |
|-------------------|-----------------|
| PyTorch 2.0 | EfficientNet-B0 |
| Transfer Learning | Grad-CAM (XAI) |
| Augmentation | Optimization |

Reasoning Layer

Large Language Model

- | | |
|--------------------|----------------|
| Claude Sonnet 4.5 | Anthropic API |
| Prompt Engineering | Context Window |
| Zero-shot Chain | JSON Output |

Interface

Web & Visualization

- | | |
|-----------------|-------------------|
| Streamlit UI | Plotly Charts |
| ReportLab (PDF) | Interactive Dash |
| Real-time Feed | Responsive Design |

Infrastructure

Data Processing

- | | |
|--------------|-----------|
| Pandas | NumPy |
| Pillow (PIL) | OpenCV |
| ETL Pipeline | Vector DB |

데이터셋 및 분석 대상 소개

Total Samples

7,348 Images

train dataset: def_front 3,758개, ok_front 2,875개

test dataset: def_front 453개, ok_front 262개

Train Set : Validation Set : Test Set = 70%(560) : 15%(120) : 15%(120)

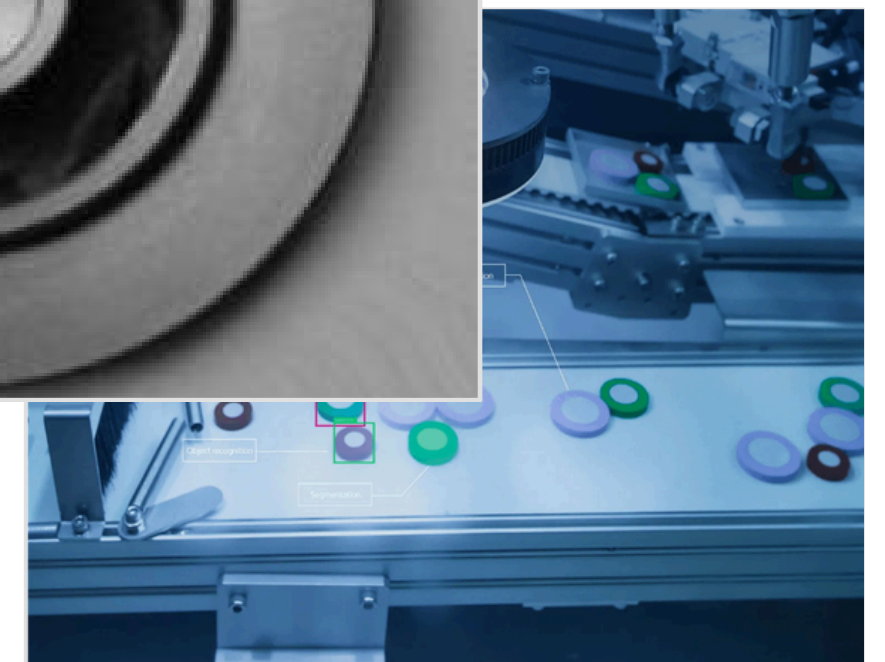
임펠러 (Impeller) 란?

모터의 회전 에너지를 유체(액체/기체)의 압력과 유속으로 변환하는 핵심 장치

산업분야: 상하수도, 석유화학, 해양선박, 식품가공 등등

임펠러와 주조의 관계

주조(Casting)는 복잡한 형상의 임펠러를 정밀하게 제작하기 위한 핵심 제조 공법



Dataset Source (Kaggle)

https://www.kaggle.com/datasets/ravirajsinh45/real-life-industrial-dataset-of-casting-product?select=casting_data

모델 비교 및 최종 모델 선정

Model Architecture	Validation Accuracy	Model Size	Inference Speed
ResNet18	99.86	44 MB	174MB/s
DenseNet121	99.16%	30 MB	162MB/s
EfficientNet-B0 	99.72%	20 MB	143MB/s
MobileNetV2	99.72%	13 MB	125MB/s

Accuracy

최고 정확도 (99.72%)

비교 모델 중 가장 정밀한 결함 탐지 성능 입증

Efficiency

경량화 (20MB)

낮은 메모리 점유율로 메모리 효율적 운영 가능

모델 구조 및 Transfer Learning



핵심 아키텍처 특징

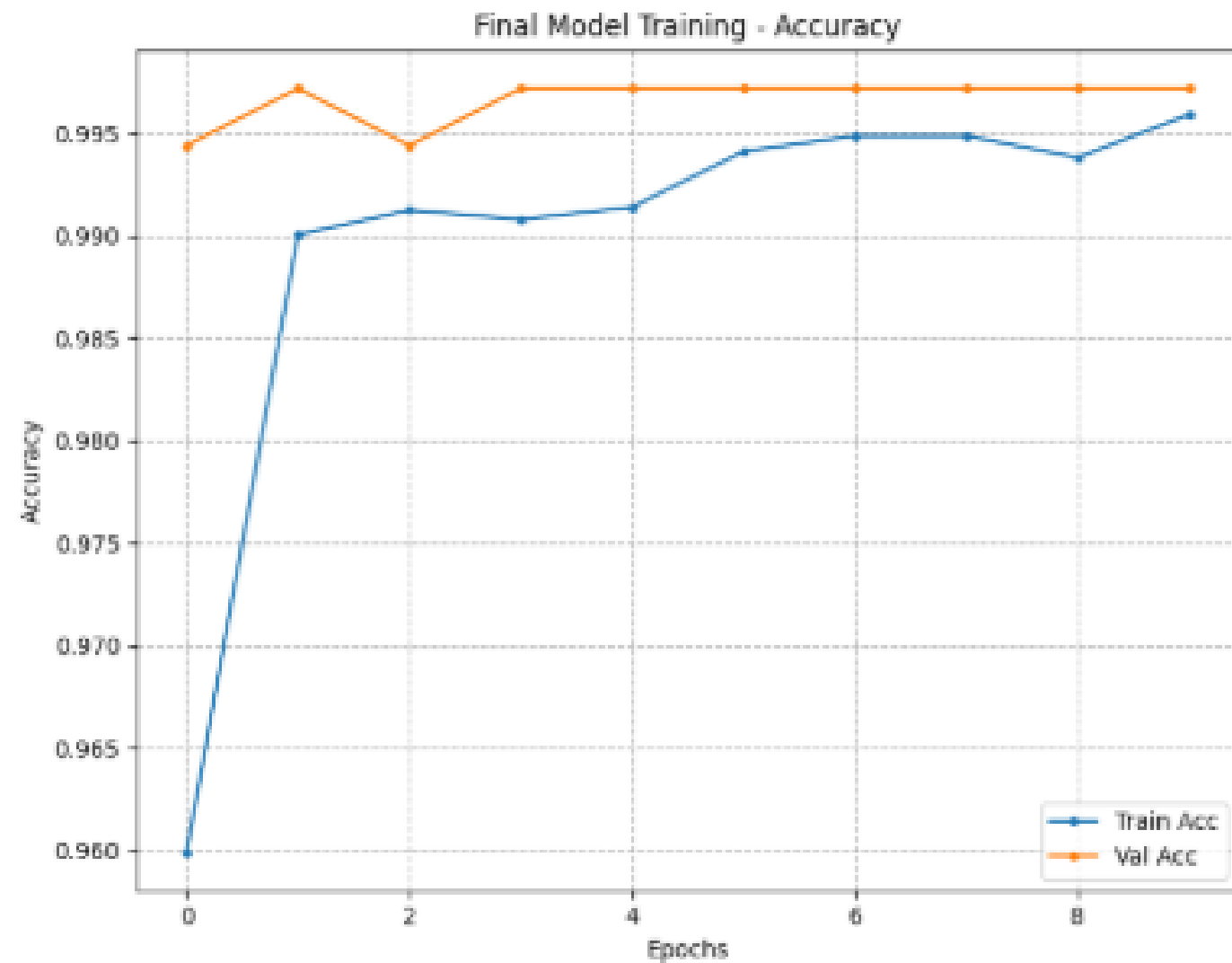
- Compound Scaling : 깊이, 너비, 해상도의 균형 잡힌 확장
- MBConv Blocks : Inverted Residual 구조의 고효율 연산
- Feature Extraction : 마지막 레이어: 1280 채널, ImageNet 사전 학습 가중치 활용

Transfer Learning Strategy

- 01 **Pre-trained Model Load**
ImageNet으로 사전 학습된 가중치 활용 (지식 전이)
- 02 **Classifier Customization**
출력층 교체 (1,000 클래스 → 2 클래스: 정상/결함)
- 03 **Regularization (Dropout)**
Dropout(0.5) 레이어 추가로 과적합(Overfitting) 방지
- 04 **Model Training**
주조 데이터(6,633장)로 전체 네트워크 학습

Feature Extractor: ImageNet 지식 재활용
Classifier: 주조 특화 학습

Accuracy & Loss 분석



Epoch1부터 Accuracy 99.44% 달성
> Transfer Learning 효과 확인

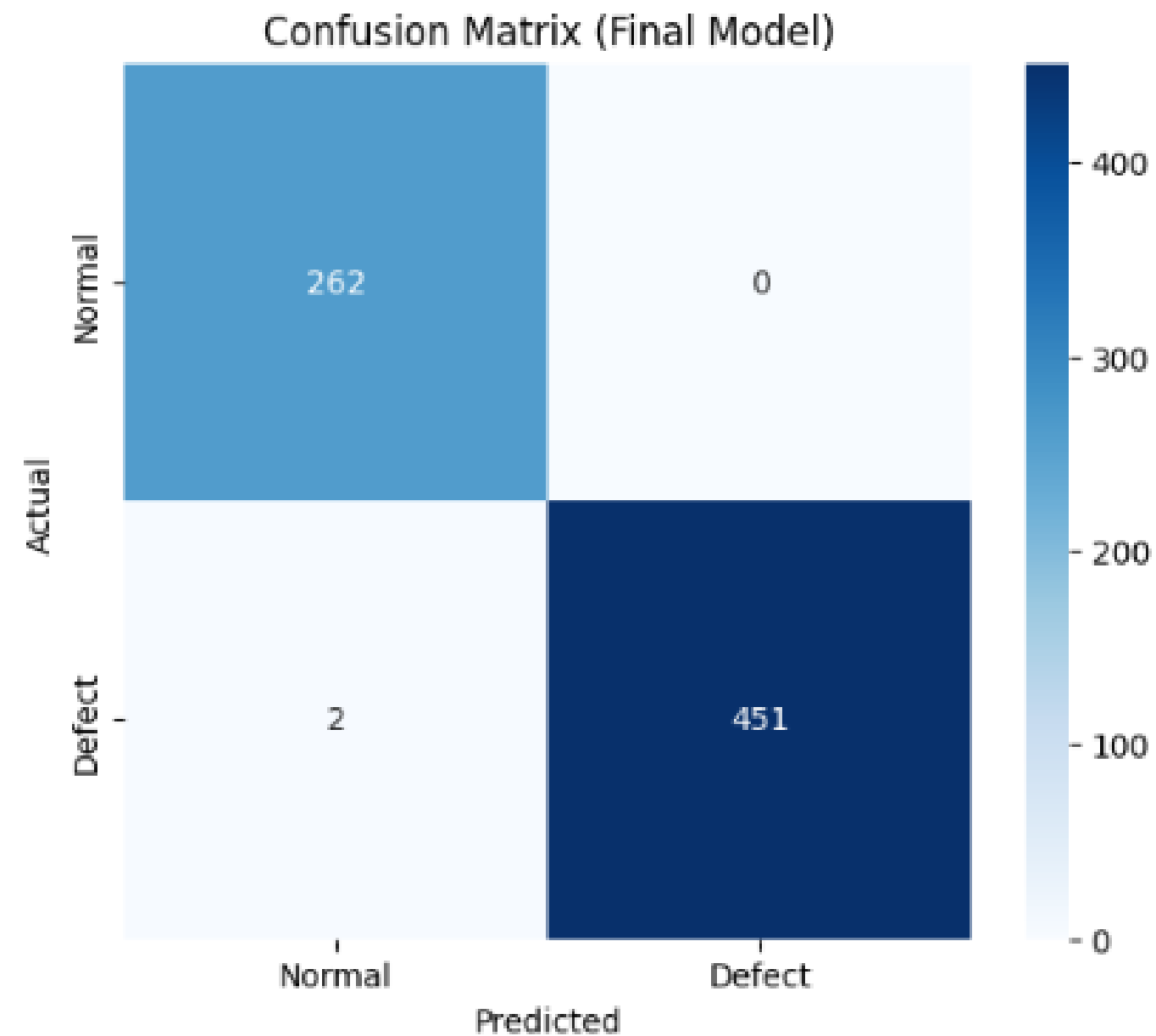


Epoch3부터 안정화 단계 진입
0.0264 → 0.0089



Train Accuracy: 99.59%
Validation Accuracy: 99.72%
- Final Loss: 0.0089

Confusion Matrix 분석



정상 제품

- 262개 중 262개 100% 분류
 - False Positive: 0개
- 👉 정상 제품을 불량으로 오판한 경우 없음!

불량 제품

- 453개 중 451개 정확히 분류 (99.56%)
 - False Negative: 2개
- 👉 불량을 정상으로 오판 (매우 낮음)

시스템 폴더 구조

```
casting_app/
├── app.py                # Streamlit UI (메인 앱)
├── config.py             # 전역 설정
├── .env                  # 환경 변수 (API 키)
├── requirements.txt      # 패키지 의존성
├── README.md             # 프로젝트 문서
├── inspection_history.csv # 검사 이력
├── services/             # 비즈니스 로직 (오케스트레이션)
│   ├── inspection_orchestrator.py # 워크플로우 관리 (123줄)
│   ├── analyzer.py         # 결함 분석 서비스
│   ├── pdf_generator.py    # PDF 보고서 생성
│   └── history.py          # 검사 이력 관리
├── classifiers/          # AI 모델 계층
│   └── image_classifier.py  # EfficientNet-B0 분류기 (44줄)
├── explainers/           # XAI (설명 가능한 AI)
│   └── gradcam.py          # Grad-CAM 히트맵 생성
├── llm/                  # LLM 통합 계층
│   ├── analyzer.py        # Claude AI 분석기 (80줄)
│   ├── client.py          # Anthropic API 클라이언트
│   └── prompt.py          # 프롬프트 빌더
├── utils/                # 유틸리티
│   └── imaging.py         # 이미지 전처리
```

```
├── models/               # 학습된 모델
│   └── efficientnet_b0_best.pth # 모델 가중치 (99.86% 정확도)
├── notebooks/            # Jupyter 노트북
│   └── casting_model_comparison.ipynb # 모델 학습 및 비교
├── assets/               # 정적 리소스
│   ├── accuracy_loss.png  # 학습 곡선
│   ├── confusion_matrix.png # 혼동 행렬
│   └── sample_*.jpeg       # 샘플 이미지
├── Font/                 # 한글 폰트
│   ├── malgun.ttf         # 맑은 고딕
│   └── malgunbd.ttf        # 맑은 고딕 Bold
└── data/                 # 학습 데이터셋 (7,348장)
    ├── train/             (70%) # 학습 데이터
    │   ├── ok_front/      # 정상 제품
    │   └── def_front/     # 불량 제품
    ├── val/               (15%) # 검증 데이터
    └── test/              (15%) # 테스트 데이터
```

Orchestration 패턴 정의

"중앙 지휘자(Orchestrator)가 여러 서비스를 순서대로 호출하고 조율하는 설계 패턴"

app.py (Conductor)

전체 흐름 통제 및 조율



분류 모듈

Violin



시각화 모듈

Cello



분석 모듈

Trumpet



PDF 생성

Piano



이력 관리

Drums

👉 비즈니스 로직이 코드로 명확히 표현

👉 에러 처리 및 복구 용이

👉 각 서비스를 독립적으로 개발/테스트

👉 순서 제어 및 조건부 실행

핵심 코드: Model & Prediction API

모델 구조 정의

```
# classifiers/image_classifier.py

def _build_model(self):
    """EfficientNet-B0 모델 생성 및 구조 변경"""
    # 1. ImageNet 사전 학습된 모델 로드
    model = models.efficientnet_b0(weights=None)

    # 2. Classifier 커스터마이징 (Transfer Learning)
    model.classifier = nn.Sequential(
        nn.Dropout(p=0.5),      # 과적합 방지
        nn.Linear(1280, 2)      # 1000개 -> 2개 (정상/불량)로 변경
    )

    return model.to(self.device).eval()
```

☞ EfficientNet-B0를 불러오고,

마지막 분류기(Classifier)를 데이터셋에 맞게(2개 클래스) 수정

Prediction API (예측 수행)

```
def predict(self, image):
    """
    이미지 예측 수행

    프로세스:
    1. 이미지 전처리 (크기 조정, 정규화)
    2. AI 모델 추론
    3. Softmax로 확률 계산
    4. 결과 반환
    """
    # 1. 이미지를 텐서로 변환하고 배치 차원 추가
    input_tensor = self.transform(image).unsqueeze(0).to(self.device)

    # 2. 그래디언트 계산 없이 추론 (속도 향상)
    with torch.no_grad():
        outputs = self.model(input_tensor)      # 모델 출력 (로짓)
        probs = torch.softmax(outputs, dim=1)[0] # 확률로 변환 (합=1)
        pred = torch.argmax(probs).item()       # 가장 높은 확률의 클래스

    # 3. 결과 딕셔너리 반환
    return {
        'prediction': pred,                      # 0=정상, 1=불량
        'confidence': probs[pred].item(),        # 예측 신뢰도 (0~1)
        'class_name': config.CLASS_NAMES[pred], # 클래스 이름
        'probabilities': {
            config.CLASS_NAMES[0]: probs[0].item(), # 정상 확률
            config.CLASS_NAMES[1]: probs[1].item()  # 불량 확률
        },
        'input_tensor': input_tensor            # Grad-CAM용 텐서
    }
```

☞ 이미지를 받아서 전처리후, 모델에 넣어서 결과를 확률과 함께 반환

핵심 코드: Grad-CAM (XAI)

히트맵 생성 로직

```
def generate(self, input_tensor, original_image):
    """
    GradCAM 히트맵 생성

    프로세스:
    1. 모델의 그래디언트 계산
    2. 중요도 맵 생성 (어디를 주목했는가?)
    3. 원본 이미지에 히트맵 오버레이

    Args:
        input_tensor: 전처리된 입력 텐서 (모델 입력)
        original_image: 원본 이미지 (PIL Image)

    Returns:
        numpy.ndarray: 히트맵이 적용된 이미지
                        (빨강=중요, 파랑=덜 중요)
    """
    # 1. GradCAM 계산 - AI가 주목한 영역 추출
    #   grayscale_cam: 0~1 사이 값 (1=가장 중요)
    grayscale_cam = self.cam(input_tensor=input_tensor)[0, :]

    # 2. 원본 이미지를 numpy 배열로 변환 (0~1 범위)
    img_array = np.array(original_image.resize(config.IMAGE_SIZE)) / 255.0

    # 3. 히트맵을 원본 이미지에 오버레이
    #   빨간색: AI가 집중한 영역 (결함 가능성 높음)
    #   파란색: AI가 무시한 영역
    cam_image = show_cam_on_image(img_array, grayscale_cam, use_rgb=True)

    return cam_image
```

Grad-CAM

(Gradient-weighted Class Activation Mapping)이란?

☞ 딥러닝 모델, 특히 CNN이 이미지의 특정 클래스를 예측할 때 어떤 영역에 집중했는지 시각적으로 보여주는 설명 가능한 인공지능(XAI) 기술

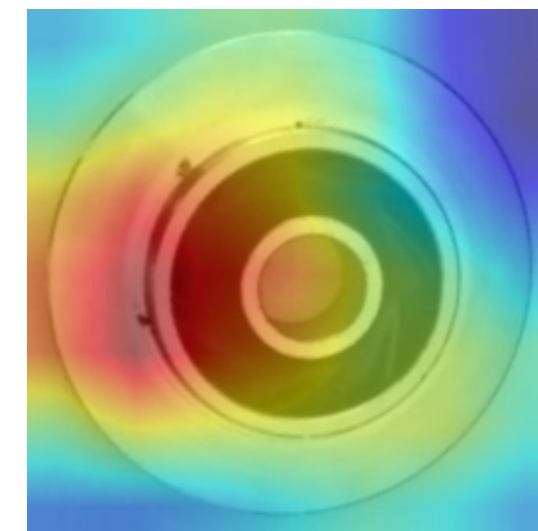
☞ 영향력 있는 픽셀 찾기 - 딥러닝 모델 분류에 가장 큰 영향을 준 픽셀 탐색

☞ 가중치 합산 - 중요한 셀만 더 가중치를 주어 합침

☞ 긍정적 영향만 추출 (ReLU)

- 불량이라고 판단하는데 긍정적인 영향을 준 부분만 남김
- 음수는 제거 → 히트맵이 붉은 이유

☞ 생성된 히트맵을 원본 이미지 위에 덮어 직관적인 UI 제공



핵심 코드: Dynamic Prompt Engineering

☞ 고정된 질문이 아닌, 검사 결과에 따른 분석 요청

```
# llm/prompt.py - 라인 27-28
```

```
normal_causes = "제품이 규격 기준을 충족한 이유를 2가지 서술"
defect_causes = "결함 발생 가능 원인을 3가지 이상 구체적으로 서술"
```

☞ python 문법인 f-string 문법을 통해
문자열 안에 변수를 {variable}형태로 넣을 수 있게 구현

```
# llm/prompt.py - 라인 23-24, 33-34
```

```
status = "정상" if prediction == 0 else "불량"
confidence_pct = f"{confidence:.2%}"
```

```
cause_instruction = normal_causes if prediction == 0 else defect_causes
action_instruction = normal_actions if prediction == 0 else defect_actions
```

```
return f"""
```

```
...
- 판정 결과: {class_name}
- AI 신뢰도: {confidence_pct}
...
```

```
### 3. 예상 원인 분석
```

```
**{status} 판정 시**: {cause_instruction}
```

```
### 4. 권장 조치 사항
```

```
**{status} 제품**: {action_instruction}
```

```
"""
```

☞ Vision AI가 찾아낸 데이터를 LLM이 이해라 수 있는 자연어 문장으로
변환해주는 '통역사' 역할

```
# llm/prompt.py - 라인 23-24, 36-45
```

```
status = "정상" if prediction == 0 else "불량"
confidence_pct = f"{confidence:.2%}"
```

```
return f"""
```

```
당신은 제조업 품질관리(QC) 전문가이자 주조 공정 전문 엔지니어입니다.
아래 AI 검사 결과를 바탕으로 **제조 현장용 전문 보고서**를 작성해주세요.
```

```
📊 AI 검사 결과
```

```
- 판정 결과: {class_name}
- AI 신뢰도: {confidence_pct}
- 분석 모델: EfficientNet-B0 (전이학습, 정확도 99.86%)
```

```
...
```

```
"""
```

핵심 코드: Workflow Control

```
class InspectionOrchestrator:
    """검사 프로세스 전체를 관리하는 오케스트레이터"""

    def __init__(self):
        """모든 필요한 모듈 초기화"""
        self.classifier = ImageClassifier()      # AI 모델
        self.explainer = GradCAMGenerator()      # 히트맵
        self.analyzer = DefectAnalyzer()         # LLM 분석
        self.pdf_generator = PDFReportGenerator() # PDF 생성
        self.history = InspectionHistory()        # 이력 관리

    def run_inspection(self, image):
        # 1. 검사 시간 기록
        inspection_time = datetime.now()

        # 2. AI 예측 수행
        result = self.classifier.predict(image)
        result['inspection_time'] = inspection_time

        # 3. Grad-CAM 히트맵 생성
        cam_image = self.explainer.generate(result['input_tensor'], image)
        result['cam_image'] = cam_image

        # 4. 검사 이력 저장
        self.history.add_record(result)

        return result
```

☞ __init__() 메서드에서 5개 모듈을 미리 준비

- AI, 히트맵, LLM, PDF, 이력 관리

☞ 순차적 실행

- run_inspection() 메소드가 4단계 실행

☞ 결과 통합

- 각 단계의 결과를 result 딕셔너리에 누적
- 최종적으로 모든 정보가 담긴 result 반환

핵심 코드: PDF 보고서 생성

```
class PDFReportGenerator:
    """주조 결함 검사 보고서 PDF 생성"""

    def __init__(self):
        self.styles = getSampleStyleSheet()
        self._setup_korean_font()    # 한글 폰트 등록
        self._setup_custom_styles()  # 스타일 정의

    def generate_report(self, result, analysis_text,
                       original_img, gradcam_img):
        """5단계로 구성된 PDF 보고서 생성"""

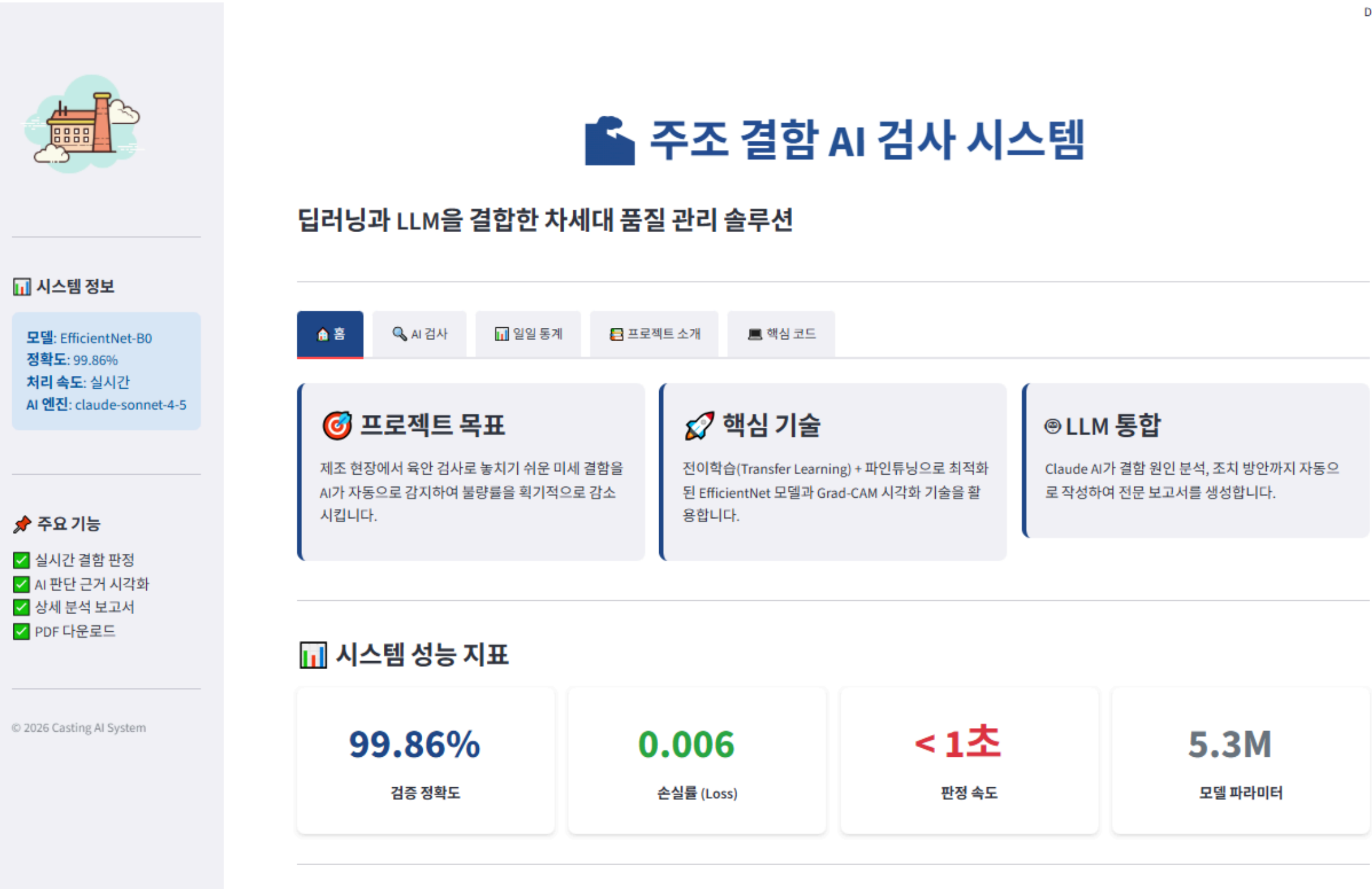
        buffer = io.BytesIO()
        doc = SimpleDocTemplate(buffer, pagesize=A4)
        story = []

        story.extend(self._create_header())           # 1. 헤더
        story.extend(self._create_inspection_info(result)) # 2. 검사 정보
        story.extend(self._create_image_analysis(
            original_img, gradcam_img))               # 3. 이미지 분석
        story.append(PageBreak())
        story.extend(self._create_ai_analysis(analysis_text)) # 4. AI 분석
        story.extend(self._create_footer())           # 5. 푸터

        doc.build(story)
        return buffer
```

- 👉 ReportLab 라이브러리를 활용한 PDF 보고서 자동 생성
- 👉 헤더, 검사정보, 이미지, AI분석, 푸터로 구성된 문서
- 👉 한글 폰트, 색상, 테이블 스타일 등 디자인 요소

streamlit 시스템 인터페이스(홈 탭)



- 시스템의 메인 대시보드 화면
- 프로젝트의 핵심 가치 (3가지)
 - 시스템 성능 지표
 - 모델 학습 결과 및 성능 분석 페이지

streamlit 시스템 인터페이스(AI 검사 탭)

정상 , 불량 샘플
각각 4개의 테스트 샘플

☞ 모델 학습을 통한
결함 검사를 하는 메인 페이지

✖ 불량 - 샘플 2

✔ 정상 - 샘플 1

✔ 정상 - 샘플 2

✔ 정상 - 샘플 3

✔ 정상 - 샘플 4

✖ 불량 - 샘플 1

✖ 불량 - 샘플 2

✖ 불량 - 샘플 3

✖ 불량 - 샘플 4

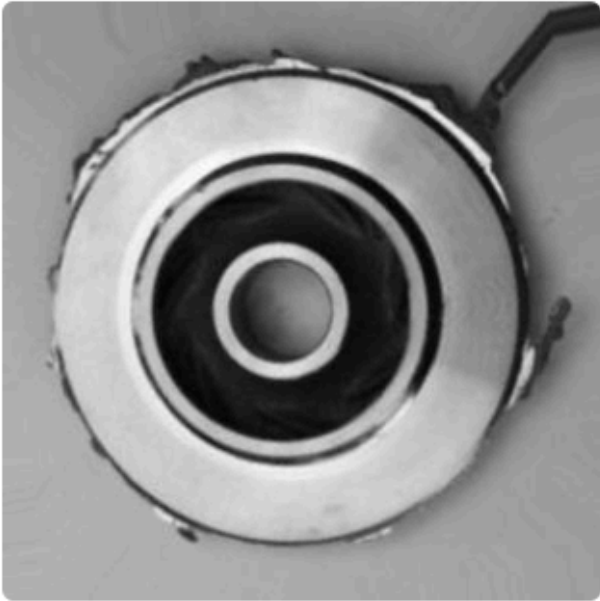
테스트할 샘플을 선택하세요

✖ 불량 - 샘플 2

✔ 선택 완료: ✖ 불량 - 샘플 2

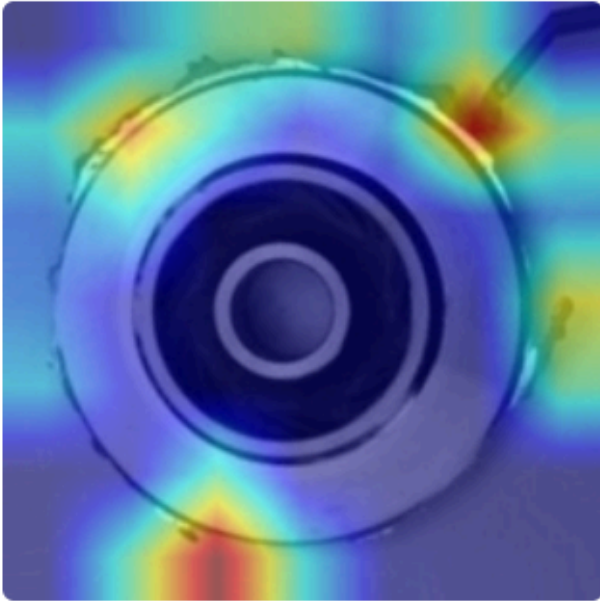
✔ 분석 완료!

📷 원본 이미지



업로드된 제품 이미지

🔥 Grad-CAM 히트맵



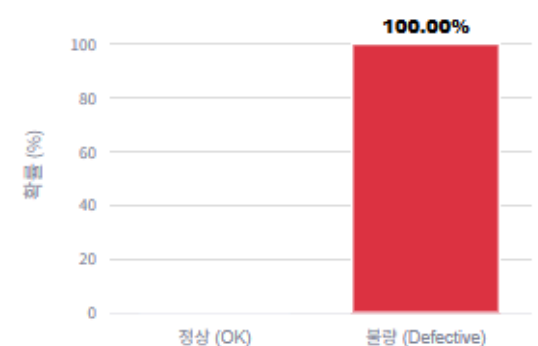
AI 판단 근거 시각화

🎯 판정 결과

✖ 불량 (Defective)

📊 클래스별 확률 분석

AI 예측 확률



클래스	확률 (%)
정상 (OK)	0
불량 (Defective)	100.00

🌈 컬러 스펙트럼 가이드

낮음
중요도: 0~30%

중간
중요도: 30~70%

높음
중요도: 70~100%

📖 해석 방법

- 빨간색: AI가 가장 집중한 영역 (결함 가능성 높음)
- 노란색: 부분적으로 주목한 영역
- 파란색: 거의 주목하지 않은 영역

🕒 검사 일시

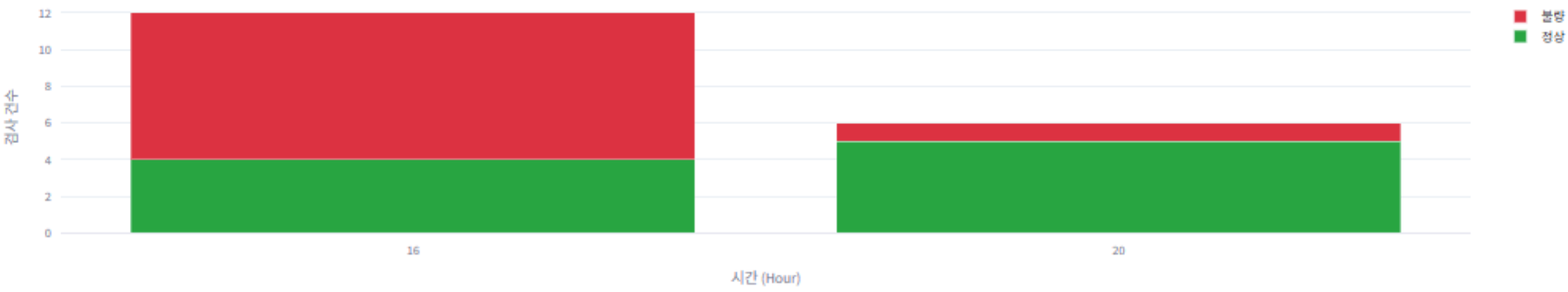
2026년 01월 12일
20시 42분 05초

컬러 스펙트럼 제공
(빨간색은 결함 가능성 높음)

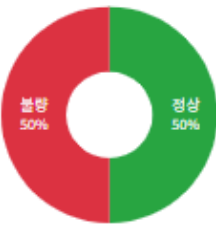
streamlit 시스템 인터페이스(일일 통계 탭)



📊 시간대별 검사 추이



📊 판정 비율



📋 최근 검사 기록

🔍 🗨

■ 정상

■ 불량

시간	판정	신뢰도
2026-01-12 20:42:05	불량 (Defective)	100.0%
2026-01-12 20:41:54	정상 (OK)	100.0%
2026-01-12 20:28:10	정상 (OK)	100.0%
2026-01-12 20:27:17	정상 (OK)	100.0%
2026-01-12 20:27:12	정상 (OK)	100.0%
2026-01-12 20:26:15	정상 (OK)	100.0%
2026-01-12 16:22:50	정상 (OK)	100.0%
2026-01-12 16:22:41	불량 (Defective)	100.0%

👉 실시간 품질 모니터링

- 실시간 검사 현황
- 시간대별 품질 추이
- 종합 판정 비율
- 상세 검사 이력 (시간 포함)

streamlit 시스템 인터페이스(프로젝트 소개 탭)

🏠 홈

🔍 AI 검사

📊 일일 통계

📁 프로젝트 소개

📄 핵심 코드

📖 프로젝트 개요

🎯 개발 배경

제조업 현장에서 주조 제품의 결함 검사는 전통적으로 숙련된 검사자의 육안 검사에 의존해왔습니다. 하지만 이러한 방식은 다음과 같은 한계가 있습니다:

- ⚠️ **인적 오류**: 피로도, 집중력 저하로 인한 불량품 누락
- 🕒 **시간 소요**: 대량 생산 시 검사 시간이 병목 구간
- 📊 **일관성 부족**: 검사자마다 판단 기준이 상이
- 💰 **인건비 부담**: 숙련 인력 확보 및 유지 비용

본 프로젝트는 AI 기술을 도입하여 이러한 문제를 해결하고자 합니다.

🔧 기술 스택

🧠 딥러닝 & AI

- PyTorch 2.0: 딥러닝 프레임워크
- EfficientNet-B0: 경량화된 고성능 CNN 모델
- Transfer Learning: ImageNet 사전 학습 모델 활용
- Fine-tuning: 도메인 특화 재학습
- Grad-CAM: 설명 가능한 AI (XAI) 기법

☁️ LLM & 백엔드

- claude-sonnet-4-5: 고급 언어 모델
- Anthropic API: LLM 연동
- Streamlit: 웹 인터페이스
- ReportLab: PDF 생성
- Plotly: 인터랙티브 차트

📊 데이터셋 & 학습

📁 데이터셋 정보

- 출처: Kaggle - Casting Product Image Data
- 총 이미지: 7,348장
- 클래스: 정상(OK) / 불량(Defective)
- 분할 비율: Train 70% / Val 15% / Test 15%

🎓 학습 전략 (Training Strategy)

- Epoch: 10 (Early Stopping 적용)
- Batch Size: 32
- Optimizer: Adam (LR=0.001)
- Pre-trained: ImageNet 가중치 사용
- 데이터 증강: 회전, 상하좌우 반전, 색상 변화

- 👉 프로젝트 소개 페이지
- 개발 배경
 - 기술 스택
 - 시스템 아키텍처
 - 데이터셋 & 학습

streamlit 시스템 인터페이스(핵심 코드 탭)

📄 **핵심 코드 설명** ↔

1 **모델 아키텍처 (EfficientNet-B0 + 파인튜닝)**

```
# classifiers/image_classifier.py

def _build_model(self):
    '''EfficientNet-B0 모델 생성 및 커스터마이징'''

    # ImageNet으로 사전 학습된 모델 불러오기 (전이학습)
    model = models.efficientnet_b0(weights=None)

    # 분류기(Classifier) 부분을 우리 문제에 맞게 교체
    # - 원래: 1000개 클래스 분류
    # - 수정 후: 2개 클래스 (정상/불량)
    model.classifier = nn.Sequential(
        nn.Dropout(p=0.5),      # 과적합 방지
        nn.Linear(1280, 2)      # 최종 출력: 2개 클래스
    )

    return model.to(self.device).eval()
```

💡 **핵심 포인트:**

- **전이학습 (Transfer Learning):** ImageNet에서 배운 익바정이 시간 트지은 환요

👉 **핵심 코드 설명 페이지**

- 모델 아키텍처 (EfficientNet-B0 + 파인튜닝)
- Grad-CAM
- LLM 통합 (Claude API로 분석 보고서 자동 생성)
- 데이터 증강 (Data Augmentation)

프로젝트 회고

느낀 점

프로젝트 초반에는 어떻게 학습을 시켜야 할지 막막했는데 프로그램의 흐름을 대략적으로 이해하고 나니 그 다음부터는 눈에 조금씩 들어오기 시작했습니다.

또한 모델을 학습시킬 때 데이터를 분리하고 학습하는 과정에서 모델 선택의 중요성을 알게되었고, Grad-CAM 의 작동 원리를 이해하고 모델에 적용시키는게 흥미로웠습니다.

이전에는 GEMINI-API 로 간단하게 챗봇을 만드는 작업을 했었는데, 좀 더 진화해서 Claude-API 를 사용하고 직접 프롬프트를 생성해봄으로써 LLM에 가까워질 수 있었습니다. 머신러닝과 딥러닝의 매력을 느낄 수 있는 프로젝트였습니다. 이를 계기로 앞으로는 좀 더 복잡한 데이터셋으로 학습을 시켜보고 싶습니다.

아쉬웠던 점

시계열 데이터보다는 0, 1로 분리되는 분류 모델에 더 흥미가 생겨 데이터셋을 구하는데 어려웠습니다. 그러던 중 마음에 드는 주제 제품 데이터셋을 발견했고 학습을 시작했지만, 의외로 단순한 이미지 패턴때문에 학습하는데 어려움이 많았습니다.

학습 과정에서 시행착오가 많았고 아직 완벽한 프로젝트라고 할 수는 없지만, 딥러닝과 LLM의 개념을 이해하는데 많은 도움이 되었습니다.

Thank You

[Any Questions or Feedback?](#)

 2026. 01

 jhwwon32@gmail.com